

STACKBURGER

Aplicación Móvil · Delivery & Takeaway

DOCUMENTACIÓN TÉCNICA

Entrega 2 — Escalado Funcional

Este documento detalla las decisiones tecnológicas y arquitectónicas incorporadas en la Entrega 2 de StackBurger: autenticación con Firebase Auth, roles de usuario, integración con Firebase Data Connect (PostgreSQL), protección de rutas, manejo de errores de red y refactoring respecto de E1.

Proyecto	StackBurger App
Framework	React Native 0.85.3
Plataforma	Android (API 24+)
Backend	Firebase Auth + Firebase Data Connect + PostgreSQL
Node.js	≥ 22.11.0

Checklist E2: ☒ Sistema de login documentado · ☒ Roles y permisos definidos · ☒ Protección de rutas · ☒ Integración con Firebase documentada · ☒ Manejo de errores de red · ☒ Refactoring respecto de E1

1. Cambios respecto de la Entrega 1

En E1 el MVP funcionaba con datos locales/simulados y un flujo lineal de pedido. En E2 se incorporó:

Área	Entrega 1	Entrega 2
Persistencia de datos	Mock / simulación local	Firebase Data Connect → PostgreSQL
Autenticación	No existía	Firebase Auth (email/contraseña)
Roles	No existían	CLIENTE y ADMIN
Catálogo	Solo hamburguesas	4 categorías: Hamburguesas, Papas, Bebidas, Postres
Pedidos	Simulados	Creación real en backend + historial del cliente
Admin	No existía	Gestión de productos y pedidos
Estado servidor	—	TanStack React Query
Navegación	Stack lineal E1	Stack ampliado + flujos cliente/admin/invitado

2. Autenticación y autorización

2.1 Sistema de login elegido: Firebase Authentication

Se eligió Firebase Auth con proveedor Email/Password por:

- Integración nativa con el ecosistema Firebase ya adoptado para datos (Data Connect).
- Persistencia de sesión en React Native mediante AsyncStorage (getReactNativePersistence).
- Escalabilidad: permite agregar otros proveedores en E3 (Google, etc.) sin cambiar la arquitectura base.
- El token JWT de Firebase se utiliza automáticamente en las operaciones de Data Connect con nivel @auth(level: USER).
-

Flujo de autenticación implementado:

Usuario ingresa email/contraseña



signInWithEmailAndPassword (Firebase Auth)



onAuthStateChanged detecta sesión activa



getMe (Data Connect) → obtiene perfil en PostgreSQL



Si no hay perfil vinculado → linkMyAccount (vincula authUid con email)



AuthContext expone user + rol → navegación según rol

Registro de nuevos usuarios:

1. createUserWithEmailAndPassword en Firebase Auth.
2. createUsuarioProfile en Data Connect con rol CLIENTE por defecto.
3. Si falla la creación del perfil, se revierte el usuario de Auth (delete + signInOut) para evitar cuentas huérfanas.

Modo invitado (sin autenticación):

- Desde Login, el usuario puede entrar como invitado (guestMode: true).
- Puede navegar el catálogo y ver detalle de productos.
- Al intentar agregar al carrito, se muestra LoginRequiredModal que lo redirige a Login/Registro.
- Componentes clave:

Archivo	Responsabilidad
AuthContext.tsx	Sesión, login, register, logout, rol
AuthGate.tsx	Pantalla de carga mientras se resuelve la sesión inicial
firebase.ts	Inicialización de Firebase App, Auth y Data Connect

2.2 Roles de usuario y permisos

Los roles están definidos en el schema de Data Connect (RolUsuario):

Rol	Valor enum	Asignación
Cliente	CLIENTE	Default en registro y usuarios seed

Administrador	ADMIN	Asignado manualmente en base de datos / seed
---------------	-------	--

Permisos por rol (aplicación):

Funcionalidad	Invitado	CLIENTE	ADMIN
Ver catálogo público	✓	✓	✓
Agregar al carrito / confirmar pedido	✗	✓	✗*
Ver historial propio de pedidos	✗	✓	✗
Gestionar productos (CRUD)	✗	✗	✓
Ver todos los pedidos del sistema	✗	✗	✓
Actualizar estado de pedidos	✗	✗	✓
Cierre de sesión	—	✓	✓

*El administrador, al iniciar sesión, es redirigido al panel administrativo y no al flujo de compra del cliente.

Redirección post-login (navigationUtils.ts):

- ADMIN → pantalla inicial AdminProducts.
- CLIENTE → pantalla inicial Home (modo autenticado).
- Invitado → Home con guestMode: true.

2.3 Pantallas

ID	Pantalla	Acceso
SBURG-E2-001	Login	Público
SBURG-E2-002	Register	Público
SBURG-E2-003	Home (catálogo)	Público (invitado) / Cliente
SBURG-E2-004	Detail	Público (invitado) / Cliente
SBURG-E2-005	Cart	Cliente autenticado
SBURG-E2-006	Checkout	Cliente autenticado
SBURG-E2-007	Orders (historial)	Cliente autenticado
SBURG-E2-008	AdminProducts	Solo ADMIN
SBURG-E2-009	AdminOrders	Solo ADMIN
SBURG-E2-010	AdminOrderDetail	Solo ADMIN

3. Integración con backend y servicios externos

3.1 Backend elegido: Firebase Data Connect + PostgreSQL

No se utiliza Firestore ni Realtime Database. El backend de E2 es Firebase Data Connect, que expone operaciones GraphQL sobre una base PostgreSQL administrada en Firebase Cloud SQL.

Ventajas para E2:

- Modelo relacional (usuarios, productos, pedidos, ítems) con integridad referencial.

- SDK autogenerado (@dataconnect/generated) con hooks React tipados.
- Queries y mutations declarativas en .gql con reglas de auth por operación.

Inicialización en la app:

```
// firebase.ts
```

```
initializeApp(firebaseConfig);
```

```
getDataConnect(app, connectorConfig);
```

La configuración sensible (firebaseConfig.ts, .firebaserc, dataconnect/) permanece fuera del repositorio; en GitHub solo se publican plantillas .example.

3.2 Estructura de la base de datos (PostgreSQL)

Entidades principales:

Usuario

```
├── id (UUID)
├── nombreCompleto
├── email (unique)
├── authUid (unique, vinculado a Firebase Auth)
├── passwordHash (solo seed/migración legacy)
├── rol: CLIENTE | ADMIN
└── createdAt
```

Producto

```
├── id (UUID)
├── nombre, descripcion, ingredientes
├── precio (Int)
├── categoria: HAMBURGUESAS | PAPAS | BEBIDAS | POSTRES
├── imagenUrl (URL externa)
├── activo (Boolean, soft-delete)
└── createdAt
```

Pedido

```
├── id (UUID)
├── codigo (unique, ej. SB-40101)
├── usuario → Usuario
├── estado: PREPARING | COMPLETED | CANCELLED
├── subtotal, costoEnvio, total
├── modalidadEntrega: DELIVERY | TAKEAWAY
├── nombreContacto, telefonoContacto, direccion
└── createdAt
```

PedidoItem

```
├── pedido → Pedido
├── producto → Producto (opcional, snapshot por nombre)
├── cantidad, precioUnitario, precioTotalLinea
├── extras: extraMedallon, extraCheddar, extraPanceta, extraCebolla
└── notas
```

Relaciones:

- Un Usuario tiene muchos Pedido.
- Un Pedido tiene muchos PedidoItem.
- Cada PedidoItem puede referenciar un Producto (se guarda también nombreProducto como snapshot).

3.3 Operaciones Data Connect utilizadas en E2

Queries:

Operación	Auth	Uso en la app
ListProductosActivos	PUBLIC	Catálogo Home (solo activo: true)
GetProductoById	PUBLIC	Detalle de producto
GetMe	USER	Perfil y rol del usuario logueado
ListMyPedidos	USER	Historial del cliente (auth.uid)
ListProductosAdmin	USER	Panel admin — productos
ListPedidosAdmin	USER	Panel admin — todos los pedidos

Mutations:

Operación	Auth	Uso en la app
CreateUsuarioProfile	USER	Registro
LinkMyAccount	USER	Vincular cuenta Auth con fila existente
CreateProducto	USER	Admin — alta de producto
UpdateProducto	USER	Admin — edición
DeleteProducto	USER	Admin — baja lógica (activo: false)
CreatePedido	USER	Checkout
CreatePedidoItem	USER	Ítems del pedido
UpdatePedidoEstado	USER	Admin — cambio de estado

3.4 Capa de datos en la app: TanStack React Query

Para E2 se incorporó `@tanstack/react-query` junto con `@tanstack-query-firebase/react` para cachear respuestas del servidor:

Parámetro	Valor	Motivo
staleTime	60 s	Reduce refetch innecesario en listas
refetchOnWindowFocus	false	Evita recargas agresivas en mobile
refetchOnReconnect	true	Actualiza al recuperar conexión

Utilidades de cache (`productQueryCache.ts`, `orderQueryCache.ts`):

- Actualización optimista/in-place tras editar productos (preserva orden en lista admin).
- Invalidación selectiva de queries tras crear pedidos o cambiar estados.

Gestión de estado — división E2:

Estado	Tecnología
Carrito	Context API + useReducer (heredado de E1)
Sesión / usuario	Context API (AuthContext)
Datos del servidor	TanStack React Query + hooks generados

3.5 Manejo de errores de red y reintentos

Autenticación (AuthContext.tsx):

Código Firebase	Mensaje al usuario
auth/invalid-credential, auth/wrong-password, auth/user-not-found	"Email o contraseña incorrecto"
auth/email-already-in-use	"Ese email ya está registrado"
auth/weak-password	"La contraseña debe tener al menos 6 caracteres"
auth/invalid-email	"Ingresa un email válido"
auth/network-request-failed	Mensaje genérico de conexión

4. Librerías nuevas incorporadas en E2

Paquete	Versión	Rol
firebase	^12.15.0	Auth + Data Connect SDK
@dataconnect/generated	local	SDK autogenerado (queries/mutations)
@tanstack/react-query	^5.101.1	Cache y estado servidor
@tanstack-query-firebase/react	^2.1.1	Integración React Query + Data Connect
@react-native-async-storage/async-storage	^3.1.1	Persistencia de sesión Auth

Se mantienen las dependencias de E1: React Native 0.85.3, React Navigation 7, TypeScript 5.8.3, etc.

5. Decisiones de refactoring

5.1 De mock local a Firebase

Antes (E1)	Después (E2)
Datos hardcodeados en constants	Queries a PostgreSQL vía Data Connect
Confirmación simulada	CreatePedido + CreatePedidoItem reales
Sin tipos de backend	Mappers en firebaseMappers.ts (Producto → Burger, Pedido → Order)
Una categoría	Filtro por CategoriaProducto en Home

Motivo: cumplir RF de persistencia, historial, admin y multi-categoría con un backend escalable sin montar servidor propio.

5.2 Separación de capas

- `services/orderService.ts`: lógica de negocio de pedidos (orquestración de mutations).
- `utils/firebaseMappers.ts`: transformación modelo DB ↔ modelo UI.
- `utils/productQueryCache.ts` / `orderQueryCache.ts`: sincronización de cache post-mutación.
- `navigation/useRoleGuard.ts`: autorización de pantallas admin.

5.3 Navegación

- Se mantiene Stack Navigator (no Tab Navigator completo aún).
 - `BottomNav` para flujo cliente (Home / Pedidos).
 - `AdminTabs` para flujo admin (Productos / Pedidos).
 - Login como `initialRouteName`; post-auth se hace `CommonActions.reset` según rol.
-
-

Resumen

La Entrega 2 evoluciona StackBurger de un MVP local a una aplicación conectada a Firebase Auth y Firebase Data Connect (PostgreSQL). Se implementaron dos roles (CLIENTE, ADMIN) con permisos diferenciados, protección de pantallas administrativas, catálogo multi-categoría, historial de pedidos, checkout con modalidad Delivery/Takeaway y panel de administración para productos y estados de pedidos. El refactoring principal consistió en reemplazar datos simulados por operaciones GraphQL tipadas, introducir React Query para el estado del servidor y organizar la app en capas (contexto, servicios, mappers, cache).

Materia: Desarrollo de Aplicaciones Móviles · 2026

StackBurger · Documentación Técnica E2 · React Native 0.85.3 + Firebase